פרק 7 - וירטואליזציה

https://www.ics.uci.edu/~aburtsev/cs5460/lectures/lecture26-virtualization-part2/lecture26-virtualizaion-part2.pdf

https://www.cs.bgu.ac.il/~os182/wiki.files/Practical%20Session%2013%20-%20Virtualization.pdf

כללי

יתרונות

איחוד שרתים, ניוד מערכת, פריסה קלה של **תוכנות תקן וירטואליות** (virtual appliances הן מכונות עם מערכת הפעלה ואפליקציות מותקנות), ניפוי שגיאות ובדיקות (במערכות הפעלה למשל), מידור כהגנה, ענן (מיקור חוץ לחברה מתמחה).

עוד מילה על virtual appliances: נועדו לעזור עם תלויות מסורבלות של תוכנות בספריות ובתוכנות אחרות. הן מאפשרות הפצה קלה של מכונות מוכנות ושירותי אפליקציה בענן.

וירטואליזציה בשכבות שונות

1. ברמת ה-ISA: חיקוי (emulation) של כל פקודות ה-Instruction Set Architecture. נתרגם לפקודות של המכונה הפיזית. איטי מאד.
2. ברמת ה- Hardware Abstraction Layer
3. ברמת מערכת ההפעלה - קונטיינרים משתפים מבני נתונים פנימיים של מערכת ההפעלה המארחת והיא מנהלת את משאבי החומרה והתוכנה ביניהם.
4. ברמת הספרייה - תרגום של קריאות מערכת לקריאת פונקציות של ספריות וירטואליזציה, כמו WINE שמאפשרת להריץ אפליקציות ווינדוס בלינוקס.
5. ברמת האפליקציה - מדברים על סביבות וירטואליות להרצת bytecode ב-run-time environment כמו .NET CLR.

וירטואליזציה ברמת ה-HAL: מימוש

מודלים של hypervisor: **טיפוס ראשון** רץ ישירות על החומרה (כמו esx), **טיפוס שני** רץ כתהליך משתמש על מערכת הפעלה מארחת (כמו virtualBox) ומסתבר שניסו פעם להוסיף טיפוס אפס שבו עיקרי הוירטואליזציה מסופקים ע"י החומרה והקוד קצר מאד.

הוראות מיוחסות (privileged instructions) הן instructions שמייצרות TRAP (פשוט software interrupt). **הוראות רגישות** (Sensitive instruction) הן instructions שההרצה שלהן מושפעת ממצב ההגנה (ring) שנמצאים בו. אם כל ההוראות הרגישות היו מיוחסות, היינו יכולים להשתמש ב-**trap and emulate**- פעולות לא מיוחסות יתבצעו באופן ישיר ופעולות רגישות תמיד יעשו trap וה-hypervisor יעשה להן חיקוי, אך ב-x86 זה לא היה המצב. דוגמא להוראה רגישה אך לא מיוחסת: הרצת POPF שיכולה לבטל

קבלת פסיקות ממצב הגנה 3 לא יצרה שגיאה אלא פשוט לא התרחשה. **מצב מיוחס וירטואלי** - כשמערכת ההפעלה האורחת חושבת שהיא רצה במצב מיוחס אך בעצם זה מצב משתמש.

הפתרון הראשון שהוצע הוא **תרגום בינארי**:
Usermode ישאר ב-3 ring של המעבד, מ"ה תעבור מ-0 ring לאחד וה-hypervisor ירוץ ב-0 ring.
כל הוראה מיוחסת תגרום ל-TRAP וה-hypervisor יטפל בה.
בנוסף, ה-hypervisor מזהה בלוקים יסודיים (רצף הוראות שמובטח שאו שהאחרונה בו רגישה או אף אחת) בזמן ריצה ואם ההוראה האחרונה בהם רגישה הוא מחליף ב-TRAP ל-hypervisor ואז ה-hypervisor יכול לטפל בה במצב מיוחס.
*אם ההוראה האחרונה משנה את ה-flow של התכנית ה-hypervisor צריך לעדכן אותה לפי השינויים שהוא ערך בבלוקים האחרים.
אפשר לשמור cache של הבלוקים המתורגמים.
יש עוד פתרון, פרא-וירטואליזציה, והוא מפורט בהמשך.

די סיוט סה"כ, אז פיתחו מעבדים מיוחדים עם טכנולוגיית וירטואליזציה - **VT**. בגדול ה-hypervisor יכול לקבוע עם bitmap חומרתי איזה הוראות מכונה יגרמו ל-TRAP.
בוירטואליזציה מטיפוס שני (מעל מערכת הפעלה מארחת), יהיו **מצבים בהם ה-hypervisor יהיה חייב להריץ קוד ב-ring0**. במצבים אלה נעשה שימוש ברכיב שרץ כחלק מה-kernel של מ"ה המארחת. מצבים בהם זה נדרש:

1. שינוי ה-segment descriptor: יש סוג של סגמנטים של מ"ה שמתוארים ע"י gate descriptors שקובעים הרשאות גישה לפי rings במעבד. נצטרך להיות ring0 כדי לערוך אותם.
2. טיפול בפסיקות: גישות לדיסק של האורחת מתורגמות לגישות לקובץ עם פורמט המדמה דיסק במארחת. הפסיקות על הפעולות יחזרו למארחת ורכיב ב-kernel של המארחת (שרץ ב-ring0) אחראי להעביר אותן ל-hypervisor.
3. טיפול ב-TRAP: באופן דומה הרכיב מזהה שקרה TRAP ואם הוא משתמש ב-trap and emulate הוא יצטרך להודיע על ה-TRAP ל-hypervisor.
4. החלפת עולם: אם התרחשה פסיקה שמתייחסת למערכת ההפעלה האורחת בזמן שהקוד של ה-hypervisor/מערכת ההפעלה האורחת/תהליך שרץ מעל מערכת ההפעלה האורחת רץ מבצעים שתי פעולות:
    a. שמירת המצב הנוכחי של מערכת ההפעלה האורחת
    b. טעינה של מצב שמאפשר קבלת פסיקות ע"יהמארחת

פרא-וירטואליזציה
פתרון וירטואליזציה שלא דורש טכנולוגיית VT במעבד - מחליפים את כל ההוראות הרגישות בפונקציות מה-API של ה-hypervisor שנקראות hypercalls.
יש דיון לגבי האם זה הופך את ה-hypervisor למיקרו-קרנל, התשובה לפי הספר: אם היינו צריכים לתמוך רק במ"ה שהחליפו את כל ההוראות הרגישות - כן, טכנית זה כבר קרוב מאד למיקרו קרנל. אבל בפועל

עלינו לתמוך גם במ"ה שמבצעות הוראות רגישות, לדעת לזהות אותן ולבצע TRAP, מה שמרחיק ממיקרו-קרנל.
בעיה: אם נחליף את כל הקריאות הרגישות ל-hypercalls איך נרוץ על חומרה ואיך נוכל להחליף ל-hypervisor של חברה מתחרה? הוצעו שני פתרונות:

1. ניסו להגדיר תקן בשם VMI (שזה virtual machine interface) - כבר לא בשימוש כי טכנולוגיות וירטואליזציה במעבדים השתפרו.
2. הוגדרה קבוצת פעולות בגרעין של לינוקס בשם paravirt ops שמאפשרת לו לרוץ גם על חומרה וגם על hypervisor עם פרא-וירטואליזציה.

וירטואליזציה של ה-RAM

הטבלה המכילה מיפוי בין המסגרות הפיזיות של חומרה במערכת האורחת למסגרות הפיזיות של החומרה האמיתית נקראת **shadow page table**.
כאשר מערכת ההפעלה האורחת עושה שינויים ב-page table שלה ה-hypervisor צריך לעדכן את ה-shadow page table אבל לא מתבצע TRAP. איך נדע על שינויים כאלה? שני פתרונות:

1. טעינה והסרה של page tables הן הוראות מיוחסות (מבצעות TRAP) לכן נדע כשהן מתבצעות ונוכל לשנות את הדפים שהטבלה נשמרה אליהם כ-read-only כך שכל גישה תגרום לשגיאה שתעלה ל-ring0 וה-hypervisor יראה אותה.
2. רק כאשר מתרחשת גישה לזיכרון ה-hypervisor בוחן את טבלת הדפים ומעדכן את המיפוי שלו.

שתי הגישות גורמות להרבה פסיקות דף והרבה מעברים מאורחת ל-hypervisor ולכן כבדות. עם פרא-וירטואליזציה הכל מתנהל עם hypercalls ולכן זה קל. כיום לא משתמשים ב-shadow page table לתרגום מכתובת פיזית של המכונה הוירטואלית לכתובת פיזית אמיתית, אלא יש תמיכה ישירה של המעבד.
מכונות וירטואליות המריצות אותה מערכת הפעלה יכולות לשתף זיכרון פיזי של מכונה מארחת. שומרים hash table ואם לשני דפים יש אותו תוכן לפי ה-hash מצביעים רק לאחד (ומשתמשים ב-copy on write).
בגלל המנגנון הזה ובגלל שהשימוש בזיכרון משתנה, יש הרבה זיכרון לא מנוצל המוקצה למכונות הוירטואליות. יש שתי דרכים לדעת באילו חלקים מהזיכרון הן באמת משתמשות:

1. נאיבית: נבנה עוד רמה של paging, כך נדע לאיזה דפים המכונה ניגשת וגם נוכל לעשות swapping.
2. התנפחות: טוענים מודול של פסאודו device driver לכל מכונה וירטואלית. ה-hypervisor דורש ממנו כמה דפים מהזיכרון של המכונה "לתפוס", כך שאם נרצה שהמכונה תשתמש בפחות זיכרון נפקוד עליו "להתנפח" ומערכת ההפעלה במכונה הוירטואלית תבחר בעצמה לאיזה pages לעשות swapping.

וירטואליזציה של I/O

כאשר מערכת ההפעלה האורחת עולה היא בודקת איזה devices מחוברים וזה יוצר TRAP. נוכל לתת לה תשובה אמיתית וכך היא תטען דרייברים מתאימים לחומרה וכאשר תנסה לגשת אליה זה יבצע TRAP ואנחנו נצטרך רק להעתיק את הפקודה. לחלופין, נוכל לשקר לה - מערכת ההפעלה תחשוב שהיא עובדת

מול חומרה X אבל אנחנו נתרגם לחומרה אמיתית שיכולה להיות גם אחרת. היתרון בגישה זו הוא שאפשר להחליף חומרה בלי לעדכן את הדרייברים של כל מערכות ההפעלה המותקנות מעל ה-hypervisor.
I/O MMU מתרגם כתובות של התקן קלט פלט לכתובות של הזיכרון הראשי. זה מאפשר מעבר נתונים ממכונה וירטואלית ישירות להתקן וגישה של התקן לזיכרון וירטואלי של מכונה נתונה בלבד. בנוסף ניתן להשתמש בו כדי למפות פסיקות למכונות הנכונות - כאשר התקן I/O מעלה פסיקה ה-I/O MMU יודע להעביר אותה למכונה הוירטואלית המתאימה.
אז ה-I/O MMU מאפשר הקצאת התקנים למכונות, אבל הוא מחייב שלכל התקן תיהיה מכונה יחידה ממופה. **SR-IOV** משלים את המנגנון - הוא מאפשר וירטואליזציה של התקנים, כך שרכיב חומרתי יחיד יוצג כמספר התקנים פיזיים שונים שמכונה וירטואלית יכולה לעבוד מולם ישירות.

ענן - מבדילים בין ציבורי, פרטי והיברידי (פרטי שבשעת עומס נעזר בציבורי).

- Infrastructure as a service
- Platform as a service
- Software as a service